PampelMuse, version 0.104

Sebastian Kamann (s.kamann@ljmu.ac.uk)

August 21, 2019

Abstract

PampelMuse is a software package with the purpose of facilitating the analysis of integral-field spectroscopic observations of crowded stellar fields. It provides several subroutines to perform the individual steps of the data analysis. The theoretical work that went into the development of this code is described in detail in Kamann et al. (2013) and not repeated here. Instead, the purpose of this document is to provide a practical guide on the analysis of IFS data with PampelMuse .

Contents

1 Getting started **3**
- 1.1 Requirements . . . 3
- 1.2 Download and Installation . . . 3
  - 1.2.1 Using the tarball – TBD! . . . 3
  - 1.2.2 Using the GIT repository . . . 4

2 The configuration **4**
- 2.1 Default configuration . . . 4
- 2.2 Modifying the configuration . . . 5

3 The input data **5**
- 3.1 Supported instruments . . . 5
- 3.2 Preparing the data . . . 5
- 3.3 The reference catalogue . . . 6
  - 3.3.1 A note about filters . . . 7

4 The PSF models **8**

5 The handling of the source coordinates **9**

6 The analysis with PampelMuse **9**
- 6.1 The PampelMuse GUI . . . 10
- 6.2 INITFIT: Selecting the sources . . . 10
- 6.3 CUBEFIT - Extracting the spectra . . . 15
- 6.4 POLYFIT . . . 17
- 6.5 GETSPECTRA - obtaining the results . . . 18
- 6.6 SUBTRES - creating residuals . . . 19
- 6.7 Recommended analysis strategy . . . 20
- 6.8 Auxiliary routines . . . 20
  - 6.8.1 SINGLESRC . . . 20

7 Troubleshooting **20**
- 7.1 Wiggles in extracted MUSE spectra . . . 20

8 Acknowledgements **21**

A The PSF inspection file **22**

1 Getting started

1.1 Requirements

PampelMuse is written in *python3*. It has been tested for version 3.5.2 but may also run with earlier *python* installations. In addition, several *python* packages are required to run PampelMuse :

- *numpy*[1], tested for version 1.12.1
- *scipy*[2], tested for version 0.19.0, ≥0.15 required.
- *astropy*[3], tested for version 1.3.2, ≥ 0.3 required.
- *matplotlib*[4], tested for version 2.0.2.
- *pandas*[5], tested for version 0.20.1.

Note that the versions of the packages listed above are just a recommendation. The code may also run with earlier versions. In addition to these packages, it is highly recommended that the following package is available on the system:

- *PyQt5*[6]

PyQt5 is required in order to work with the graphical user interface (GUI) that is provided with PampelMuse . When starting PampelMuse the availability of *PyQt5* is checked. If the package is not available, the interactive mode of will be disabled.

1.2 Download and Installation

There are currently two possibilities to obtain PampelMuse that will be described in the following.

1.2.1 Using the tarball – TBD!

A zipped version of the code is available at `http://path.to/pampelmuse.tar.gz`. After the download is completed, change to the folder containing the tarball and execute the following commands

```
# tar -xvzf PampelMuse-X.Y.Z.tar.gz
# cd PampelMuse-X.Y.Z/
# python setup.py install
```

This will install PampelMuse to your default *python* path. Note that the last step usually requires root privileges. If you do not have them, you can define a custom path using the `--prefix` or `--home` arguments. For example, the following command will install PampelMuse to folders `$HOME/lib/python` and `$HOME/bin`:

```
# python setup.py install --home=~
```

Afterwards you have to make sure that the system finds the newly installed files by adding `$HOME/bin` to your `PATH` and `$HOME/lib/python` to your `PYTHONPATH`. For a bash shell, this is done by adding

```
# export PYTHONPATH=$PYTHONPATH:$HOME/lib/python
# export PATH=$PATH:$HOME/bin
```

to your shell configuration file (`$HOME/.bashrc`). For a c-shell, the following commands lines should be added to your configuration file (`$HOME/.cshrc`):

```
# setenv PYTHONPATH $PYTHONPATH:$HOME/lib/python
# setenv PATH $PATH:$HOME/bin
```

For further information on custom installation paths, check the *python* documentation.[7]

Afterwards, you should be able to execute

```
# PampelMuse --help
```

If no error message is returned, you have installed PampelMuse successfully!

[1] `http://www.numpy.org/`
[2] `http://scipy.org/`
[3] `http://www.astropy.org/`
[4] `http://matplotlib.org/`
[5] `http://pandas.pydata.org/`
[6] `http://www.riverbankcomputing.co.uk/software/pyqt/intro`
[7] `http://docs.python.org/2/install/#alternate-installation`

1.2.2 Using the GIT repository

The maintenance of PampelMuse is handled via an GIT repository hosted at `https://gitlab.gwdg.de`. The advantage of using the repository is that it allows you to always use the most recent version of the code. However, the disadvantage is that when updating the code I sometimes commit revisions that are not fully operational or have freshly introduced bugs. So when updating to a new revision, be sure to check the most recent log entries.

To install from the GIT repository, first cd into the folder were you want to install the code (like `~/tools/`) and clone the repository:

```
# cd ~/tools/
# git clone https://gitlab.gwdg.de/skamann/pampelmuse.git
```

This should download the most recent version of PampelMuse . In order to use the code afterwards, the following modifications should be added to your shell configuration. For a bash shell, use

```
# export pampelmuse_path=<path to PampelMuse>/ ## ~/tools/pampelmuse in the example above
# export PYTHONPATH=$PYTHONPATH:$pampelmuse_path
# export PATH=$PATH:$pampelmuse_path/bin
```

and for a c shell use

```
# setenv pampelmuse_path <path to PampelMuse>/ ## ~/tools/pampelmuse in the example above
# setenv PYTHONPATH $PYTHONPATH:$pampelmuse_path
# setenv PATH $PATH:$pampelmuse_path/bin
```

Note that the GIT repository also contains the setup script. So you can also execute

```
# python setup.py install
```

and proceed in analogy to Sect. 1.2.1. The downside of this approach is that you will have to reinstall PampelMuse every time you pull a new version from the repository.

In either case, calling

```
# PampelMuse --help
```

should return no error message.

2 The configuration

2.1 Default configuration

The behaviour of PampelMuse can be modified using a number of configuration parameters. The default configuration that is shipped with the code is contained in an JSON[8]-file that is stored under `$pampelmuse_path/pampelmuse/config/data/pampelmuse.json`. To view this file, open it with your favourite text editor, e.g. via

```
# emacs $pampelmuse_path/pampelmuse/config/data/pampelmuse.json
```

In the following sections, the individual parameters of the configuration will be gradually introduced when discussing the part of PampelMuse that they influence. Note that each parameter is introduced via the statement

section|**key** type=*type* default=value comment

that provide its name, type (e.g. string or float), default value, and a (hopefully) helpful comment about its usage. Parameters that influence the overall behaviour of PampelMuse are listed in the section *global*, some of which are explained in the following.

global|**verbose**	type=*integer*	default=3	The chattiness of PampelMuse , i.e. the amount of output produced on the console, increasing from 0 to 4. The code uses the logging module of python, the different numbers correspond to the following levels: 0–CRITICAL, 1-ERROR, 2–WARNING, 3–INFO, 4-DEBUG

global|**cpucount**	type=*integer*	default=1	The number of CPUs used by the parallelized parts of PampelMuse . If set to −1, the code uses all available CPUs on the system.

[8] see `https://en.wikipedia.org/wiki/JSON` for an introduction

2.2 Modifying the configuration

I strongly recommend leaving the JSON-file untouched even if you want to change the configuration. Instead, make use of the following to supply your own set of configuration parameters to an analysis.

When starting any subroutine of PampelMuse , you will always have to provide a user-configuration file which should also be in plain JSON format. The user-configuration file may only contain sections and keywords available in the default configuration. Note that some parameters are left blank in the default configuration (like the prefix of the IFS data file to be analysed) and hence must be provided in the user-configuration file. Below is an example of a valid file provided by a user aiming to extract stellar spectra from MUSE data of 47Tuc.

```
  {
    "global": {
        "prefix": "DATACUBE_FINAL",
        "cpucount": 1
    },
    "catalog": {
        "name": "ngc104_starlist.csv"
    },
    "psf": {
        "fit": 30,
        "fwhm": 4.0
    },
    "sky": {
        "binsize": 50
    }

}
```

3 The input data

3.1 Supported instruments

In general, PampelMuse is designed to correctly handle IFS data regardless of which instrument was used to observe the data. However, because of the huge diversity of data formats that are produced by the individual IF spectrographs, I cannot guarantee that it will work out of the box for each instrument. So far, I successfully tested it using data from

- PMAS (Roth et al., 2005)
- ARGUS (Pasquini et al., 2002)
- MUSE (Bacon et al., 2010)
- KMOS (Sharples et al., 2004)

The routine will try to figure out which instrument the data is coming from by searching for the standard header keyword `INSTRUME`. For some instruments (like PMAS) this keyword is not set. In such cases, if you want to make use of the instrument-specific features of PampelMuse (see Sect. 3.2), you should update the FITS header of your data accordingly.

If you have data from another instrument that is not properly handled by PampelMuse , let me know. On the other hand, if the data is handled correctly, that would be very helpful for me to know as well.

3.2 Preparing the data

First note that PampelMuse is *not* a data reduction pipeline. All analysis steps assume that the IFS data has been properly reduced and that all the instrumental artefacts have been removed. In general, the IFS data can come in different formats depending on which instrument was used in the observations:

Type	required?	EXTNAME
Data	y	DATA
Variances	y	STAT, ERROR, SIGMA
Bad pixel mask	n	MASK

Table 1: Extension names used to identify the input required for the analysis in a combined FITS file.

1. As a three-dimensional data cube with two spatial axes (NAXIS1 and NAXIS2) and a spectral axis (NAXIS3). This format can always be used, even if the data were obtained with an instrument that is unknown to PampelMuse .

2. As a two-dimensional "image" with one spatial and one spectral axis. The advantage of this format is that it can also be used with instruments that have a ireegular (i.e. non-rectangular) spatial sampling. However, for this format to work PampelMuse must know how to arrange to different spaxels in the field of view. Currently this format is only supported for PMAS and ARGUS data.

3. **New in version 0.90**: MUSE data may also be provided in the form of a pixtable as it is produced by the pipeline recipe `muse_scipost` when the option `--save individual` is being used.[9] The analysis of MUSE pixtables with PampelMuse requires the data to be sorted by increasing wavelength. This can be achieved by running the dedicated `PTSORT` routine before starting the analysis.

Besides the actual data, PampelMuse also requires an estimate of the variances for the analysis. Optionally, a bad pixel mask can be used. Data, variances, and mask can either be provided as different extensions of a single FITS file or as three individual FITS files. In the former case, the routine will use the extension names to identify what is stored in which extension, see Table 1. In the latter case, the individual filenames should have the same prefix and be suffixed `.dat.fits` (the data), `.err.fits` (the variances), and `.msk.fits` (the bad pixel mask).

If provided, the bad pixel mask should be zero for valid pixels and one for pixels that should be excluded. Instead of providing a dedicated mask, one can also set the invalid pixels to NaN and PampelMuse will ignore them in the analysis.

Although not required, it is highly advantageous if the header of the FITS file containing the IFS data contains some basic WCS information, i.e. at least the keywords `RA`, `DEC`, and the matrix `CDi_j`. PampelMuse will use this information to determine the spaxel scale and the approximate location of the field of view on the sky.

PampelMuse also offers the possibility to analyse only a part of the data cube or to bin the data in spectral direction to speed up the analysis. To this aim, the section *layers* has been introduced in the configuration. It contains the following keys.

layers|first type=*integer* default=0 — The first layer of the IFS data that should be analysed. Negative numbers are counted backwards from the last layer.

layers|last type=*integer* default=−1 — The last layer of the IFS data that should be analysed. Again, negative numbers are counted backwards from the last layer.

layers|binning type=*integer* default=1 — The spectral binning factor applied to the IFS data during the analysis.

3.3 The reference catalogue

As PampelMuse does not currently include a dedicated source detection algorithm, the analysis relies on the presence of a reference catalogue, containing coordinates and magnitudes of the stars in and around the observed field. The best sources for such a catalogue are imaging data obtained with the *Hubble* space telescope (HST) or ground based adaptive optics images. Note that the quality of the reference

[9] For details, check the MUSE pipeline manual available at the ESO homepage: `https://www.eso.org/sci/software/pipelines/muse/`.

catalogue (especially its completeness and astrometric accuracy) will strongly influence the results of the analysis. For example, a common shortcoming of catalogues generated from HST data is that the brightest stars are often missing because they were saturated in the exposures. If this is the case, try to add them manually.

For PampelMuse to recognize the reference catalogue, its file name should be provided in the *catalog* section of the configuration.

catalog|name	type=*string*	default=	The name of the astrometric reference catalogue used for the analysis. **IMPORTANT:** This parameter must be set in any user-configuration file.

The reference catalogue should be present as a plain csv-file with the first (non-commented) line containing the column names. PampelMuse will recognize column names such as `ra`, `dec`, or `id` and process the column content accordingly. The reference catalogue must contain the following columns:

1. A unique identifier for every source; must be a non-negative integer of not more than 10 digits.
2. Two columns containing the source coordinates in a reference system. Both pixel and world coordinates are accepted, and priority is given to the latter if both are present. If provided, the world coordinates should be in units of decimal degrees.
3. A magnitude for every source in a passband that covers at least some part of the spectral range of the IFS data. Currently the filter name also needs to be specified in the configuration, using the parameter

catalog|passband	type=*string*	default=f814w	The name of the filter in which the magnitudes provided in the reference catalogue are given.

Here is an example of how a valid reference catalogue file may look like (but note that a fifth column labelled "status" is required when using the SINGLESRC routine instead of INITFIT):

```
id,ra,dec,f606w
1,12.764311,-26.986152,13.29
2,12.751325,-26.986431,14.87
4,12.760123,-26.976514,12.25
...
```

In some occasions, it may happen that the brightest stars from the reference catalogue appear saturated in the IFS data. In such cases, it is advised to flag these stars using the *saturated* key of the *catalog* section. As input, it takes a tuple containing the ID(s) of the saturated star(s). PampelMuse will then ignore these stars when determining the PSF of the observation.

catalog|saturated	type=*tuple*	default=	The list of stars from the reference catalogue that appear saturated in the IFS data.

3.3.1 A note about filters

PampelMuse comes with a limited number of filters. Currently, the transmission curves for *F555W*, *F606W*, *F775W*, *F814W*, and *K* are provided. If the input photometry uses another filter, it is possible to provide PampelMuse with the required information. To this aim, set the environment variable `PMFILTERPATH`. It must point to the folder in which the transmission curve of the filter is stored, e.g.

```
# export PMFILTERPATH=$HOME/data/filters.
```

The transmission curve of the filter must be available as a plain ascii file, containing at least two columns. The first column must contain the wavelength in units of Angstrom, the second column must contain the transmission in absolute values, i.e. ranging from 0 to 1. The name of the ascii file must be the name of the filter in capital letters. For example, if you want to use the SDSS g filter, the file should be named G or SDSS_G, but not g.txt or sdss_g. In the reference catalogue, you should use the same label for the photometry as you use for the filename. The parameter `PASSBAND` must also be set accordingly.

4 The PSF models

PampelMuse currently uses purely analytical PSF profiles. The user can choose between a Gaussian and a Moffat profile. An extensive discussion of the profiles and all associated formulae can be found in Appendix B. Here we restrict the discussion to the main formulae and most important properties. The analytical profile of the Gaussian is

$$G(r) = \Sigma_0 \exp\left\{\frac{-r^2}{2\sigma^2}\right\}, \tag{1}$$

whereas the Moffat profile is described via

$$M(r) = \Sigma_0 \left[1 + \left(\frac{r}{r_\mathrm{d}}\right)^2\right]^{-\beta}. \tag{2}$$

For either profile, Σ_0 is the central intensity. The width of the Gaussian is controlled by the standard deviation σ, while the effective radius r_d defines the width of the Moffat. However, instead of σ and β, PampelMuse uses the FWHM as a measure for the profile width (see Appendix B). Note that the FWHM is always measured in units of spaxels.

The Moffat profile has a second shape parameter, β. It measures the kurtosis of the profile, i.e. the strength of the wings of the PSF. Note that higher values of β correspond to less pronounced wings. From my experience, $\beta \sim 2.5$ is a typical value for a seeing-limited PSF. For this value, a Moffat profile has much more pronounced wings than a Gaussian profile. Therefore I strongly recommend using the Moffat profile because the Gaussian profile will almost certainly underestimate the wings of the PSF. Since version 0.100, PampelMuse also supports combinations of the two profiles, namely a double Gaussian and a Moffat-Gaussian profile.

The radius r incorporates the ellipticity e and the position angle θ of each profile. Thus, besides the actual centroid coordinates, the Moffat profile has 4 free parameters, the Gaussian profile has 3.

The properties of the the PSF during the analysis are controlled via the *psf* section of the configuration. The profile in use can be set using the *profile* keyword.

psf|profile type=*string* default=moffat The name of analytical PSF profile to be used. Currently supported are 'moffat', 'gauss', 'double_gauss', and 'moffat_gauss'.

The initial values of the PSF parameters can be specified directly in the *psf*-section of the configuration.

psf|e type=*float* default=0.0 The initial value of the ellipticity of the PSF.

psf|theta type=*float* default=0.0 The initial value of the position angle of the PSF.

psf|fwhm type=*float* default=3.0 The initial value of the FWHM of the PSF.

psf|beta type=*float* default=2.0 The initial value of the beta-parameter of the PSF (only used with a Moffat profile).

psf|f12 type=*float* default=0.5 The initial value of the fractional flux in the first component in case a PSF profile with two components is used.

psf|fwhm1 type=*float* default=3.0 The initial value of the FWHM of the first component in case a PSF profile with two components is used.

psf|fwhm2 type=*float* default=6.0 The initial value of the FWHM of the second component in case a PSF profile with two components is used.

Furthermore, it can be controlled which of the PSF parameters should be optimized during the fit and which ones should be held constant at their initial values. This is controlled via the *fit* keyword of the *psf* section of the configuration.

psf|fit	type=*integer*	default=6	The PSF behaviour in the fit. PSFFIT = $\sum_n 2^n$, where $n \geq 1$ runs over the indices of the free parameters; e.g. PSFFIT = 6 for a Moffat means to fit β and FWHM at constant e and θ.

Finally, the radial extent of the PSF profiles can be controlled with the following key.

psf|radius	type=*float*	default=10	The centroid distance out to which the profile will be calculated, given in spaxels. Must of course be > 0.

The PSF radius should be chosen large enough so that no rings of positive residual flux appear around the brightest stars in the residuals.

5 The handling of the source coordinates

To obtain IFS spaxel coordinates (x^n, y^n) from the coordinates (u^n, v^n) of source n in the reference catalogue, it is currently assumed that a global affine transformation exists that can be described by

- a scaling by a factor ξ along the u-axis,
- a scaling by a factor ν along the v-axis,
- a rotation around an angle θ,
- shifts along the x- and y-axis of the IFS cube.

In summary, we thus have the following formulae:

$$x^n = \xi \cos\theta \, u^n - \nu \sin\theta \, v^n + x_0 \,, \tag{3}$$

$$y^n = \xi \sin\theta \, u^n + \nu \cos\theta \, v^n + y_0 \,. \tag{4}$$

Substituting $A = \xi \cos\theta$, $B = -\nu \cos\theta$, $C = -\nu \sin\theta$, and $D = \xi \sin\theta$ we obtain

$$x^n = A \, u^n + C \, v^n + x_0 \,, \tag{5}$$

$$y^n = D \, v^n - B \, u^n + y_0 \,. \tag{6}$$

The parameters A to y_0 of the coordinate transformation can be optimized in the analysis. Their behaviour during the extraction is controlled via the parameter *posfit* in the *initfit* section of the configuration.

initfit|posfit	type=*integer*	default=2	Defines how the coordinate transformation is handled during the analysis: 0 – no optimization; 1 – only optimize the shifts, i.e. parameters x_0 and y_0; 2 – optimize all 6 parameters of the transformation.

Note that older versions of PampelMuse only used 4 parameters to determine the coordinate transformation, which is the special case of an affine transformation where $\xi = \nu$.

6 The analysis with PampelMuse

PampelMuse provides three main routines that perform the source extraction: INITFIT, CUBEFIT, and POLYFIT. They are described in the following. There are also several auxiliary routines available for various other tasks. To get an overview of all the available routines, type

```
# PampelMuse -h
```

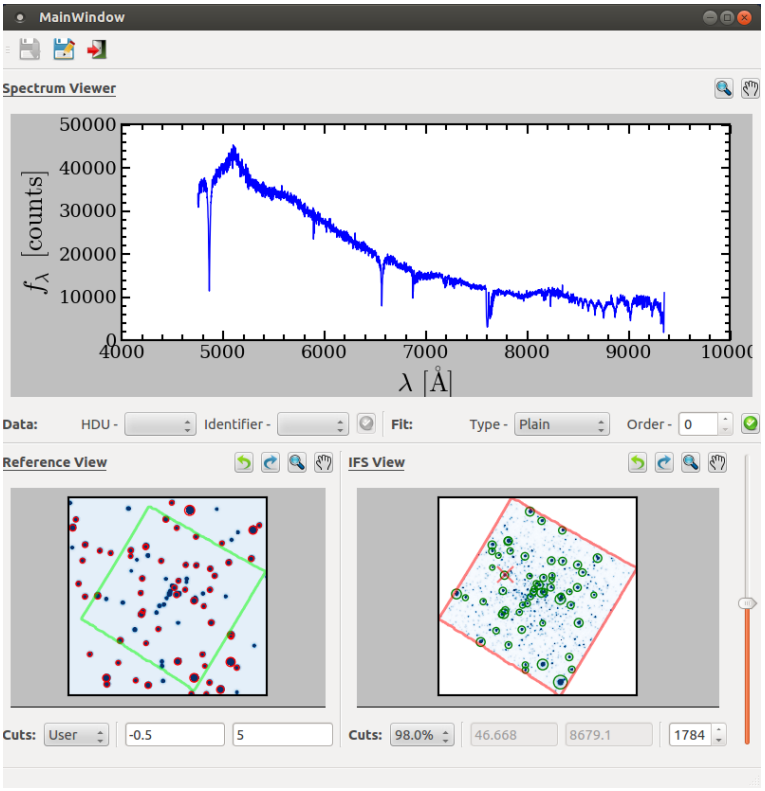


Figure 1: The graphical user interface (GUI) of PampelMuse . The bottom right panel shows a MUSE datacube of NGC 6397 observed during commissioning. The spectrum of the spaxel marked by a red cross is shown in the upper panel. The lower left panel shows a mock image created from a photometric catalogue.

6.1 The PampelMuse GUI

Before describing the individual routines of PampelMuse , the graphical user interface (GUI) that is part of several routines is introduced. The GUI is loaded automatically when running PampelMuse in interactive mode and executing the INITFIT (Sect. 6.2) or the POLYFIT (Sect. 6.4) routine. An example of how the GUI looks like is displayed in Fig. 1.

The GUI shows three separate panels. The upper half is reserved for the spectrum viewer while the lower half contains the reference viewer on the left and the IFS viewer on the right side. The buttons to the top right of each panel allow to zoom in to and out of the data that is displayed, pan over it, and recover previous views of it. The cuts used to display the reference and the IFS data can be changed using the buttons at the bottom of either panel. Finally, the slider to the right of the IFS viewer can be used to scroll through the different layers of the IFS data.

Note that the GUI will only appear if PampelMuse is executed in interactive mode. This is controlled by the parameter *interact* in the *global* section of the configuration.

global|interact	type=*boolean*	default=True	If True, the code will open a GUI that allows the user to control several steps of the data analysis. Otherwise, these steps are performed without any user interaction. Note that the automated mode requires that the basic WCS information mentioned in Sect. 3 is available in the header of the IFS data.

6.2 INITFIT: Selecting the sources

The first step of the analysis is to identify those sources from the reference catalogue for which it is feasible to extract spectra from the IFS data. As the reference catalogue ideally was obtained from

higher resolution data and typically covers a larger area on the sky, it would not make much sense to try and extract a spectrum for each star in the catalogue. Instead, we use INITFIT to make a selection. The syntax for calling this routine is

```
# PampelMuse <user configuration file> INITFIT .
```

The parameter `user configuration file` refers to the user-specific setup as explained in detail in Sect. 2. Note that the IFS data to be analysed must be provided in the *global* section of the user-configuration file, as key *prefix*. PampelMuse expects it without the `.fits` or `.dat.fits` suffix.

INITFIT performs the following steps in order to select a useful subset of the available sources.

1. Opening the IFS data, checking their validity.
2. Opening the reference catalogue file, checking its validity.
3. Creating an image from the IFS data in the passband in which the magnitudes in the reference catalogue were provided.
4. Creating a mock image from the sources in the reference catalogue, again in the passband in which the magnitudes were provided. The mock image is prepared using the same pixel size as the IFS data and the initial guess of the PSF for the IFS data (see the keys in the *psf* section of the configuration, Sect. 4).
   Note: Depending on the coverage of the reference catalogue and on the number of sources in it, this step could take a lot of time and/or consume a lot of memory. To limit both memory usage and computing time, the code will omit fainter sources and those that are too far away from the predicted footprint of the IFS data. If you still encounter that too much memory is being used, try reducing the *ccmaxoff* key in the *initfit* section of the configuration (see below).
5. Cross-correlating the two images just created to find the most likely location of the IFS field of view with respect to the reference catalogue. Note that this step requires that both images indeed have the same pixel scale and orientation. To ensure this, RA and Dec coordinates of the sources must be provided with the reference catalogue and the header of the IFS data must include the WCS information mentioned in Sect. 3. Otherwise this step is skipped.
   One issue that can be encountered when doing the cross-correlation is that a wrong pointing solution is selected because the cross-correlation signal is dominated by the brightest stars in the reference catalogues. To suppress this behaviour, the parameter *ccmaxoff* in the *initfit* section can be used.

initfit|ccmaxoff	type=*float*	default=-1.0	The maximum allowed offset with respect to the header WCS position when locating the IFS pointing via cross correlation with the photometric catalogue, given in spaxels. The default, −1.0, sets the allowed offset to half the size of the field of view. If set to 0, the pointing suggested by the header WCS will be left unchanged.

   Note that PampelMuse stores the offsets determined via cross-correlation in two `HIERARCH` keywords, `PAMPELMUSE INITFIT CCXOFF` and `PAMPELMUSE INITFIT CCYOFF`. The values provided there give the shifts along the x- and y-axis of the IFS data in units of spaxels. In order to correct the WCS information of the cube for the offsets with respect to the reference catalogue, subtract the shifts from the values of `CRPIX1` and `CRPIX2`.
6. Loading the PampelMuse GUI with the content following content (also in Fig. 1):
   - The *Reference view* shows the mock image created from the photometric input catalogue. The sources are marked with red circles and are pickable. When selecting a source via the left mouse button, the GUI displays some information about it in the status bar at the bottom of the GUI. If the cross correlation in the previous step was successful, then the reference view also shows the suggested position and orientation of the IFS field of view.

- The *IFS view* shows a layer from datacube (initially the central one). If the cross-correlation in the previous step was successful, then the GUI will show the suggested positions of the brighter sources as green circles. It is possible to grab individual spaxels from the cube using the left mouse button.
- The *Spectrum viewer* is initially empty. If a spaxel is grabbed in the IFS view, then the spectrum of that spaxel is displayed.

Note that this step is skipped if PampelMuse is run in non-interactive mode.

7. From here, there are basically two options on how to proceed:
   (a) If the cross correlation was successful and you are happy with the location of the IFS field of view with respect to the reference catalogue, then you can just close the GUI and let INITFIT proceed with the source selection.
   (b) If the cross correlation failed or you are not happy with its result, then you will have to define the approximate location of the IFS field of view with respect to the reference catalogue manually. To do so, proceed as follows. Pick a source in the reference catalogue, then select its approximate position in the datacube. Once a source is successfully identified, it will be displayed by a filled red circle in the Reference view. To undo the identification for a source, pick it with the *right* mouse button in the Reference view. At least three sources should be identified before closing the GUI. INITFIT will then use this information to re-determine the location of the IFS field of view.

   In non-interactive mode, the code assumes that the location of the IFS field of view determined via cross-correlation is correct.

8. Knowing the positions of the sources in the IFS field of view, INITFIT will then estimate the signal-to-noise ratio (S/N) for all sources from the reference catalogue. To this aim, it assumes a photometric zeropoint in the provided filter, calculates the summed flux of all sources inside the field of view and compares it to the actual flux in the image created from the IFS data (cf. step 3). The photometric zeropoint is then adapted to bring the two fluxes in agreement. If instead a fixed photometric zeropoint should be used (not recommended!), the key *zeromag* of the *initfit* section may be used.

initfit|zeromag	type=*float*	default=	Sets the photometric zeropoint of the IFS data in the provided passband, i.e. the magnitude of a source creating 1 count/layer. If set to null (recommended), the zeropoint is estimated from the data.

   Depending on the size of the reference catalogue, this step may take some time, even though it is parallelized.

9. The next step is the actual source selection. INITFIT will loop over the sources in the reference catalogue – sorted by magnitude, i.e. brightest first – and do a number of checks to determine the status of every source in the fitting process:
   - Check if the local source density of brighter sources is below the provided threshold, defined by the key *confdens* in the *initfit* section of the configuration. As for a larger field of view, such as the MUSE WFM, the stellar density can be very different across its extent, the confusion limit is estimated locally in an area of radius $10 \times$ FWHM around each star.

initfit|confdens	type=*float*	default=0.4	The confusion density, i.e. the maximum density of resolvable sources per resolution element. The local resolution limit will be the magnitude at which the density of brighter sources reaches this density. The default value of 0.4 was determined using simulations (see Kamann et al., 2013).

   - Check if S/N is above the threshold value, defined by the key *snrcut*.

initfit|snrcut	type=*float*	default=3.0	The minimum signal to noise of sources that are included in the extraction.

Note that the S/N estimated in the previous step is modified in two cases.

(a) If a source is outside the IFS field of view, the S/N is penalized by $\exp(-(d/\mathrm{FWHM})^2)$, where d is the distance to the nearest spaxel.

(b) If a source is nearby a brighter source (with a distance $r < \mathrm{FWHM}$), the S/N is modified with the factor $1. + 0.9\log(r/\mathrm{FWHM})$. This factor approximates the decrease in S/N found for nearby sources in Kamann et al. (2013).

- Check that there is no brighter source in the direct vicinity, defined by the configuration key *binrad*, of the source.

initfit|binrad	type=*float*	default=0.3	The minimum spatial separation (in units of the PSF FWHM) of a source that is considered resolvable to a brighter source.

Sources that passed all the above checks are assigned status 2. They are considered resolved and their spectra are set for extraction. Sources that fail the last criterion or the S/N criterion because of the vicinity of a brighter source are given status 1 and their contribution is accounted for in the extraction of the spectrum of the brighter neighbour. All other stars are given status -1 and are ignored during the spectrum extraction.

10. The next step is only carried out if an unresolved stellar component should be included in the extraction process. The idea here is that the sources that are fainter than the resolution limit still create a grainy structure across the field of view (similar to surface brightness fluctuations in nearby galaxies). If the reference catalogue is of sufficient depth, this graininess can be modelled using the broadband magnitudes, positions, and the PSF. In the extraction, a combined spectrum will be extracted for the entity of these sources. If such a component is used and which sources enter it is defined by the keys in the *unresolved* section of the configuration.

unresolved|use	type=*boolean*	default=False	If True, the routine will collect all sources inside the IFS field of view that are less than *binsize* magnitudes fainter than the faintest resolved source and extract a combined spectrum for them.

unresolved|binsize	type=*float*	default=2.0	The width of the magnitude interval in the sense that the faintest sources included in the unresolved component are *binsize* magnitudes fainter than the faintest resolved sources. Only used if *use* is True.

All sources in the unresolved component are given status 0.

11. To account for the spectrum of the night sky or an unresolved stellar background, additional components may be included in the extraction. To this aim, the key *use* of the *sky* section of the configuration should be enabled. The way background components are currently handled can be summarized as follows. A homogeneous grid is defined across the field of view, its sampling is controlled by the key *binsize* of the *sky* section. Each grid point defines an additional component in the optimization. The weight of a spaxel in a grid point is obtained from an inverse distance weighting, where distance is measured using a two-dimensional Gaussian with width 0.5×`binsize` centred on the grid point.

sky|use	type=*boolean*	default=True	If True, one or more non-stellar components are included in the fit to account for background sources.

sky|binsize	type=*float*	default=-1	The distance between two background patches. If set to -1, only a constant background is used.

As a rule of thumb, in crowded fields with a significant number of unresolved sources, *binsize* should be of the order of the spatial scale over which the stellar density changes significantly. As an example, Fig. 2 compares the residuals of the extraction for a MUSE cube of NGC 6266 when using a constant or a variable sky. It is clear that the constant sky cannot account for the strong increase in stellar density towards the cluster centre. For this reason, the stars leave systematic positive residuals in the outskirts (where the sky is overestimated) and negative residuals in the centre (where the sky is underestimated).

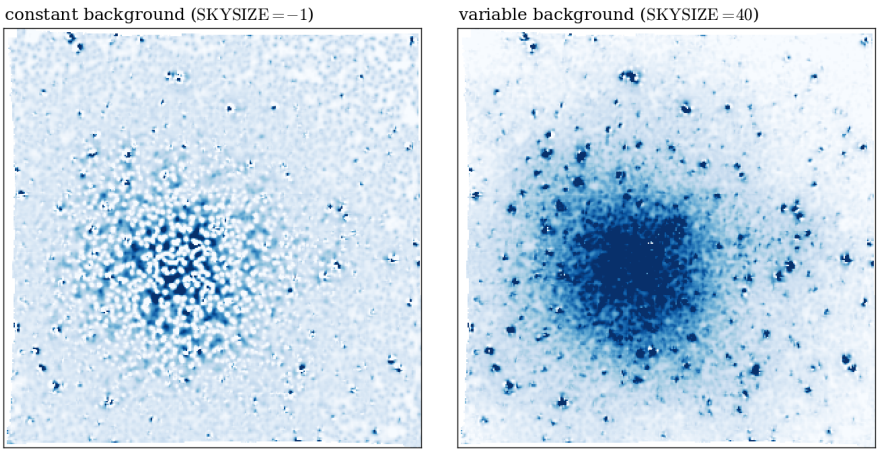


Figure 2: Illustration of improper background handling, using MUSE commissioning data of NGC 6266. The left panel shows the residuals of the source extraction when using a constant background, the right panel shows the residuals with a variable background.

12. The next step depends on the size of the IFS data, more precisely on the number of spaxels. PampelMuse was designed to handle data coming from various instruments, in particular PMAS and MUSE. Now, PMAS has 256 spaxels while MUSE has more than 90 000. As a consequence, the way the PSF is optimized for the two instruments is different. Within a field of view as small as the one of PMAS, you will almost never find reasonably isolated and bright stars that allow you to constrain the PSF. Instead, for PMAS-like data the idea is to fit a PSF profile to all resolved sources simultaneously. While this approach works well for ~ 20 sources which you typically resolve in a PMAS cube, it is computationally very demanding for the $\sim 5\,000$ sources you expect in a MUSE cube. However, for MUSE data the chances of finding good PSF stars is much higher. To account for this diversity in IFS data, PampelMuse uses the key *smallifu*, defined in the *global* section of the configuration.

global|smallifu	type=*bool*	default=False	This flag defines whether all resolved sources are used to fit the PSF and the coordinate transformation (if True) or only a subset of reasonably bright and isolated PSF sources.

My recommendation is to set *smallifu* to True for cubes with less than $\sim 2\,000$ spaxels and to False otherwise. In the latter case, INITFIT will search for suitable sources to model the PSF during the extraction. Potential PSF sources are those that fulfil the following criteria:

- at least 3 magnitudes brighter than measured resolution limit
- identified as resolved (i.e. status 2) in the source selection
- an estimated S/N > 20
- a distance $\geq$ FWHM from the edge of the IFS field of view
- two isolation criteria:

EXTNAME	Type	Purpose
PRIMARY	Primary	Header contains the configuration of PampelMuse
SOURCES	Binary Table	The source catalogue, contains basic information about every source used in the extraction.
SPECTRA	Image	Contains the extracted spectra and an estimate of the uncertainties.
POSPARS	Binary Table	Contains the coordinate transformation from reference to IFS coordinates.
POSITIONS	Binary Table	Contains the parameters of the polynomial fits to the IFS coordinates.
PSFPARS	Binary Table	Contains the parameters of the fitted PSF

Table 2: The different extensions of the PRM-file and their respective contents.

- The relative contribution from neighbouring sources inside the region where the PSF is modelled (see key *radius* of configuration-section *psf*) must be smaller than the value of key *apernois*, defined in the *initfit* section of the configuration.

initfit|apernois	type=*float*	default=0.1	The maximum allowed contamination around a PSF source from a neighbouring source. It is measured as the flux ratio of the brightest source within a distance PSFRAD and the flux of the PSF star itself.

- No brighter sources are allowed inside 1.5× `PSFRAD` distance.

Aperture photometry is performed on each layer for each source that fulfils the above criteria. The aperture radius is defined by the key *aperrad* in the *initfit* section of the configuration.

initfit|aperrad	type=*float*	default=3.0	The radius in spaxels of the circle around each PSF source in which the aperture photometry is performed.

Note that not only the fluxes are measured, but also the first and second order spatial moments. After that, the resulting spectra are used to redetermine the S/N estimate of each source and to perform the final selection of PSF sources. Also, the centroids of the aperture photometry are used to estimate the source coordinates as a function of wavelength. Finally, the second order moments measured for the PSF sources are used to estimate the ellipticity and the position angle of the PSF (if they are free parameters in the PSF fit, see Sect. 4).

13. Write the results from the source selection to a dedicated multi-extension FITS-file, named <*prefix*>.*prm.fits* and hereafter called PRM-file. This file is loaded by all other routines of PampelMuse . An overview on the structure of the PRM-file is given in Table 2.

6.3 CUBEFIT - Extracting the spectra

CUBEFIT is the routine that actually performs the extraction of the spectra by simultaneously fitting a PSF to all sources that have been identified as resolvable by INITFIT. The calling sequence for CUBEFIT is

```
# PampelMuse <user configuration file> CUBEFIT .
```

Note that the routine expects a file PRM-file generated by INITFIT. This file also stores the user-configuration that was provided to INITFIT in the header of the primary HDU. However, a user configuration file must still be provided and contain at least the *prefix* key to point PampelMuse to the IFS data that should be analysed. If a modified configuration compared to INITFIT is provided to CUBEFIT (or any other routine that requires the PRM-file), preference is given to the parameter values in the provided configuration. However, only a subset of the configuration parameters can be changed

after running INITFIT. In general, all parameters that influence the source selection (such as keys of the *psf*, section) cannot be changed any more.

CUBEFIT analyses the IFS data layer by layer, starting from the central layer and simultaneously proceeding into the red and the blue part of the cube. On each layer, the code performs an iterative analysis in order to extract the optimal flux for each source. This iterative scheme, also depicted in Fig. 3, can be summarized as follows.

1. Using the initial guesses for the PSF and the coordinate transformation, a first fit of the fluxes is performed. The flux fit is a linear least-squares fit that can be solved by direct matrix inversion and does not require initial guesses. The matrix contains the PSF profiles of the individual components. As each PSF profile is significantly different from zero only in the vicinity of the represented star, the majority of the matrix entries are zero. For such sparse matrices very efficient inversion algorithms are available. CUBEFIT uses the method `lsmr` (Fong and Saunders, 2010) from the `scipy.sparse.linalg` package. It was tested for up to 10 000 sources and provided reliable solutions in reasonable computation times.

2. With an initial value for all fluxes available, CUBEFIT updates the coordinate transformation from reference to IFS coordinates via the Levenberg-Marquardt algorithm `leastsq` from the `scipy.optimize` package. To speed up the computation, CUBEFIT internally calculates the Jacobi matrix, i.e. the matrix containing the partial derivatives of the PSF profile(s) with respect to the fitted parameters of the coordinate transformation, also provided in App. B.

3. The third step, the optimization of the PSF parameters, uses the fluxes and the updated coordinate transformation. Again, the code makes use of the `leastsq` algorithm and incorporates the Jacobi matrix, this time containing the partial derivatives with respect to the PSF shape parameters.

4. The updated PSF and coordinate transformation are used to refit the fluxes of the sources. Afterwards, it is checked whether the fluxes converged. The convergence criterion that is used by PampelMuse is that the median change in flux compared to the last iteration is < 0.005. If the convergence criterion is fulfilled or the maximum number of iterations has been reached, the analysis is continued with the next layer. Otherwise the iteration continues with fitting the coordinate transformation parameters (step 2). The maximum number of iterations can be set by the key *iterations* in the *cubefit* section of the configuration.

cubefit|iterations	type=*integer*	default=10	The maximum number of iterations that is performed by CUBEFIT on an individual layer of the IFS data.

5. After all the requested layers have been analysed in this way, the results are written to the PRM-file.

Note that the initial after the first two layers have been processed, the code will use the fit results for the PSF and the coordinate transformation as initial guesses for the analysis of the next layer. To avoid that a failed fit in one layer (e.g. because of a strong cosmic) influences the analysis of the following ones, the initial guesses are obtained by averaging over the results from the 10 adjacent layers.

It should be highlighted again that the way the fit of the coordinate transformation and PSF parameters is carried out depends on the value of the parameter `SMALLIFU`, cf. Sect. 6.2. If the value of `SMALLIFU` is True, the code optimizes for the parameters by fitting a PSF profile to all resolved sources simultaneously. Otherwise, the code loops over all PSF sources found by INITFIT and fits a single PSF profile to them. Afterwards, the parameters of the coordinate transformation are obtained using the reference coordinates and the fitted centroids of all PSF sources. The PSF parameters are updated by averaging over the results from the individual PSF fits. To remove contamination from neighbouring sources as accurately as possible, those sources are subtracted from the IFS data before the fit using the current PSF estimate.

Note that this routine is computationally the most demanding part of PampelMuse . The executions times can vary from a few minutes if only some sources are present to tens of hours for really crowded fields and huge datasets. My recommendation is to start CUBEFIT in a `screen` session, detach from it and have a good glass of wine. (Note however that for budget reasons, the wine is not shipped with PampelMuse .)

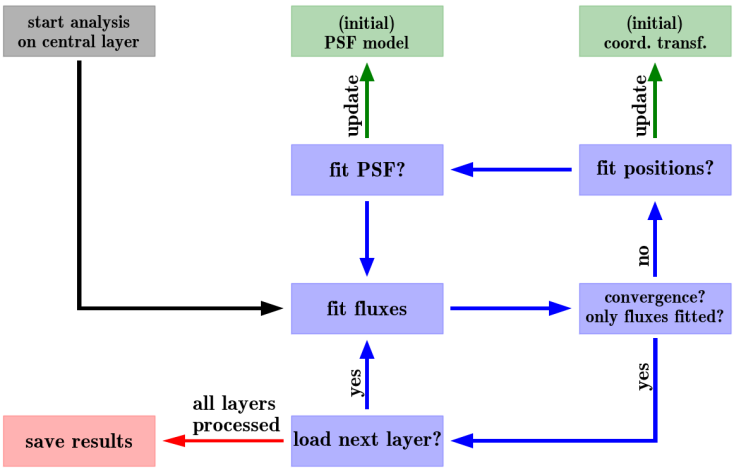


Figure 3: The iterative fitting approach of the CUBEFIT routine.

6.4 POLYFIT

After running CUBEFIT, you can check the results using the PampelMuse GUI. To this aim, start the POLYFIT routine via

`# PampelMuse <user configuration file> POLYFIT`. This routine serves two purposes. On the one hand, it allows for a visual inspection of all fitted parameters, i.e. the extracted spectra, the IFS coordinates, and the PSF. On the other hand, it can be used to fit wavelength dependencies of the PSF parameters and the IFS coordinates with polynomials. The idea behind the latter is that from a theoretical point of view, you expect those quantities to vary smoothly with wavelength, e.g. because of differential atmospheric refraction (DAR). But because of the limited S/N in each layer, the fitted variation with wavelength will show scatter, especially in spectral regions were the signal is low like telluric absorption bands. Fitting those variations with polynomials averages out the scatter and should in principle lead to a higher S/N in the extracted spectra. When starting CUBEFIT again after performing the polynomial fits, the routine will only refit the fluxes in each layer while keeping the other parameters fixed at their polynomial values.

When running PampelMuse in interactive mode, the routine will load the GUI with the content that is displayed in Fig. 4. In large parts it looks similar to the one that was opened by INITFIT. In the following the functionality of the GUI is explained.

- The purpose of the Spectrum Viewer is to display the wavelength dependencies of the fitted parameters of spectra. The data to be shown can be selected using the two drop-down menus in 'Data:' to the bottom left of the panel. Via 'HDU', you can select the parameter class (spectrum, coordinate, PSF parameter) to be displayed, while 'Identifier' lets you select the ID in case that a spectrum or coordinate should be displayed or the specific parameter in case the PSF or the coordinate transformation should be displayed. The requested data is displayed upon clicking the green button.
  Using the left mouse button, you can exclude wavelength ranges from the polynomial fits. To this aim, keep the mouse button pressed while scanning over the desired range. Doing the same with the right mouse button lets you unmask regions. If any wavelength ranges should be excluded from the fits by default (e.g. because of strong telluric absorption), you may specify these ranges using the *mask* key in the *polynom* section of the configuration. This is helpful especially if POLYFIT is run in non-interactive mode (see below).

polynom|mask	type=*tuple*	default=	In case any wavelength range(s) should be excluded from the polynomial fits by default, provide their start and end values. The values should be provided as a single tuple of even length, so that even indices contain the start values and odd indices the end values of the ranges to be masked.

The two drop-down menus in 'Fit:' to the bottom right of the panel allow you to specify the type and degree of the polynomial that is used to fit the data. Their default values are controlled by the keys *type* and *degree* in the *polynom* section of the configuration file.

polynom|type	type=*string*	default=plain	The type of polynomial used to fit the wavelength dependencies. Can be one of 'plain' (normal polynomial), 'chebyshev', 'legendre', and 'hermite'.

polynom|degree	type=*integer*	default=0	The degree of the polynomials used to fit the wavelength dependencies.

The fit is carried out via a click on the green button to the right. Note that currently only fits to the PSF parameters and the IFS coordinates are supported. Fitting the individual IFS coordinates instead of the coordinate transformation (which was used to calculate them) is preferred because the parameters of the coordinate transformation can sometimes show correlated variations (in particular if the field of view contains only a few spaxels) that are difficult to describe with polynomials.

- The IFS View displays again the cube, this time with the positions of the extracted sources overplotted. The size of the circles used to mark the extracted sources sclae with their brightness. Sources marked by green circles are used to fit the PSF. While this display is very helpful for small fields of view, it can look very messy for MUSE-like data of crowded stellar fields. However, you can use the magnifying glass to the upper right of the panel to zoom into the data. When clicking on a source while the Spectrum View shows a spectrum or an IFS coordinate, the spectrum/coordinate of the selected source will be shown.
- The Reference View is currently not used.

After you have fitted all the desired parameters and are happy with the results, you can use the buttons to the top left of the GUI to save the results to the PRM-file.

Note that when PampelMuse is not run in interactive mode (see key *interact* in *global* section), POLYFIT will not show the GUI but use polynomials of requested order to fit the source coordinates and the free PSF parameters automatically. The type and order of the polynomials being used as well as the wavelength ranges that should be masked can still be controlled via the respective configuration keys (see above).

6.5 GETSPECTRA - obtaining the results

Once the analysis is complete, you probably want to have the spectra in some other format than the PRM-file. To facilitate this, PampelMuse has a routine GETSPECTRA, called with the following sequence.

```
# PampelMuse <user configuration file> GETSPECTRA .
```

The routine will store all the extracted spectra in individual FITS files that contain the spectrum in the primary extension and an estimate of the uncertainties in an image HDU named `SIGMA`. The header of each file will contain the correct dispersion information, general information about the star and the spectrum as well as a number of HIERARCH keywords copied from the PRM-file. Note that this routine requires the *prefix* key to be set in the *getspectra* section of the user configuration file. This is a string (typically the name of the observed cluster, galaxy, ...) that is used as a common prefix to all FITS files that are written. Additionally, the key *folder* in the same section can be used to provide a folder in which all spectra should be stored.

getspectra|prefix	type=*string*	default=	The common prefix given to all FITS files that are written.

getspectra|folder	type=*string*	default=spectra	The name of the folder in which the FITS files should be stored.

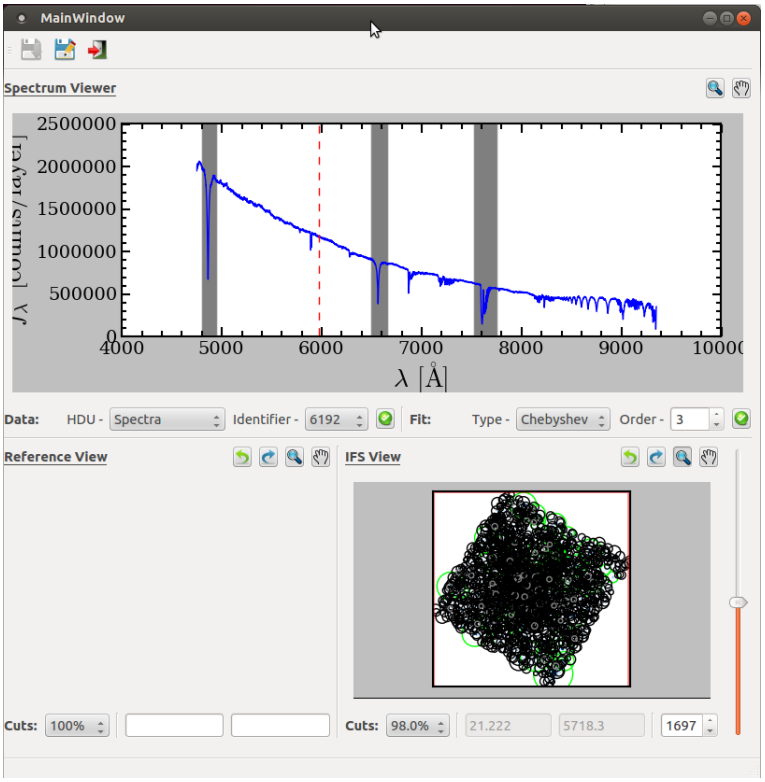


Figure 4: The GUI that is loaded by the POLYFIT routine when using PampelMuse in interactive mode.

6.6 SUBTRES - creating residuals

PampelMuse also offers the possibility to create a residuals cube once the analysis is complete. To this aim, the routine SUBTRES can be used. By default, it will subtract the contribution of all resolved stellar sources from the cube. However, it is also possible to subtract a subset of the resolved sources, using either of the *include* or *exclude* keys in the *subtres* section of the configuration. Only one of the keys should be set at a time.

subtres|include	type=*array*	default=	The list of IDs of resolved sources that should be subtracted from the IFS data using SUBTRES. The default is to subtract all resolved sources unless the option *exclude* is used.

subtres|exclude	type=*array*	default=	The list of IDs of resolved sources that should NOT be subtracted from the IFS data using SUBTRES. By default, all sources are subtracted unless the sources to subtract are specified using the *include* option.

In addition, SUBTRES allows the user to subtract the sky background or the background created by an unresolved stellar component (if either of the components has been used in the analysis). To this aim, a key *subtract* exists in both the *sky* and *unresolved* sections of the configuration.

sky|subtract	type=*boolean*	default=False	Flag indicating if the sky background should be removed when running the SUBTRES routine.

unresolved|subtract	type=*boolean*	default=False	Flag indicating if the unresolved stellar background should be removed when running the SUBTRES routine.

6.7 Recommended analysis strategy

The strategy I usually use when analysing IFS data is the following. After selecting the sources for which spectra should be extracted via INITFIT, I perform a first extraction with CUBEFIT in which the PSF parameters and the coordinate transformation are optimized across the whole wavelength range with a moderate binning in spectral direction, `layers|binning` ~ 50. Afterwards, I use POLYFIT to smooth the wavelength dependencies of those parameters via polynomial fits. Finally, I run CUBEFIT again, this time without any spectral binning. Since the PRM-file now contains the polynomial fits to the positions and PSF parameters, it will only fit the source fluxes without any iteration. Hence, the second run of CUBEFIT should be much faster than the first.

6.8 Auxiliary routines

6.8.1 SINGLESRC

The routine SINGLESRC can act as a replacement for the INITFIT routine for fields where no reference catalogue is available. It should currently only be used for non-crowded fields with reasonably isolated sources. The calling sequence for this routine is

```
# PampelMuse <user configuration file> SINGLESRC .
```

As INITFIT, SINGLESRC requires the *name* key in the *catalog* section to be set. The catalogue should be in the same format described above for the INITFIT routine, i.e. a plain csv-file. Besides the source IDs and estimated magnitudes in the filter provided as key *passband*, it should contain columns x and y, containing the source coordinates. However, in contrast to INITFIT, SINGLESRC assumes that the coordinates provided in the file are actual IFS spaxel coordinates, not reference coordinates. The routine will also not do any source selection, but use all of the provided sources in the analysis. To this aim, the catalogue should contain an additional column, named 'status', which should be 2 for sources for which the flux and coordinates are fitted and 3 for sources that are additionally used to fit the PSF parameters.

SINGLESRC will perform aperture photometry for every provided source on each layer of the IFS data to improve the initial guesses for the source coordinates. Afterwards, the routine will prepare a PRM-file that can be used to start the following routines, i.e. CUBEFIT and POLYFIT. In contrast to the procedure described earlier, CUBEFIT will not constrain a coordinate transformation but directly fit the IFS coordinates of each source.

7 Troubleshooting

7.1 Wiggles in extracted MUSE spectra

When running PampelMuse with MUSE data, some extracted spectra may show wiggles in the continuum (see Fig. 5). The reason for this is that by default PampelMuse uses the variances provided by the MUSE pipeline (extension `STAT`). However, the final reduction step — resampling of the data onto a homogeneous grid — shifts power into the covariances. As the covariances are neglected by the pipeline, the variances show similar wiggles. While formally the variances are correct for individual pixels, they underestimate the true ones as soon as one starts to combine pixels. But that is what PampelMuse does, hence the wiggles appear. As a workaround, you can switch of the usage of the uncertainties during the extraction with the parameter `USESIGMA`.

cubefit|usesigma	type=*float*	default=True	Flag indicating if the uncertainties provided with the IFS data are used in the extraction.

If the uncertainties should not be used, the code tries to estimate them from the model. To this aim, the standard deviations of the residuals after subtracting the model from the data are determined as a function of the model flux. In any case, make sure to carefully check the quality of the extracted spectra afterwards. In some cases, it may be necessary to smooth the uncertainties outside of PampelMuse prior to the extraction.

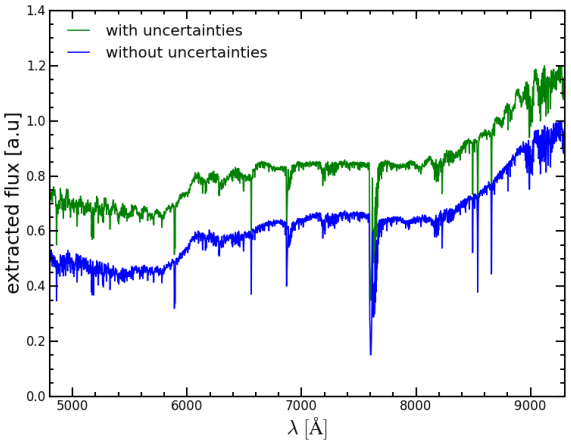


Figure 5: Comparison of spectra extracted from a MUSE cube of NGC6266 with (green) and without (blue) using the uncertainties provided by the data reduction pipeline. Note that the spectra have been offset for clarity.

8 Acknowledgements

Tim-Oliver Husser, Benjamin Giesers, Bruno Dias, Anja Feldmaier, ...

References

R. Bacon, M. Accardo, L. Adjali, H. Anwand, S. Bauer, I. Biswas, J. Blaizot, D. Boudon, S. Brau-Nogue, J. Brinchmann, P. Caillier, L. Capoani, C. M. Carollo, T. Contini, P. Couderc, E. Daguisé, S. Deiries, B. Delabre, S. Dreizler, J. Dubois, M. Dupieux, C. Dupuy, E. Emsellem, T. Fechner, A. Fleischmann, M. François, G. Gallou, T. Gharsa, A. Glindemann, D. Gojak, B. Guiderdoni, G. Hansali, T. Hahn, A. Jarno, A. Kelz, C. Koehler, J. Kosmalski, F. Laurent, M. Le Floch, S. J. Lilly, J.-L. Lizon, M. Loupias, A. Manescau, C. Monstein, H. Nicklas, J.-C. Olaya, L. Pares, L. Pasquini, A. Pécontal-Rousset, R. Pelló, C. Petit, E. Popow, R. Reiss, A. Remillieux, E. Renault, M. Roth, G. Rupprecht, D. Serre, J. Schaye, G. Soucail, M. Steinmetz, O. Streicher, R. Stuik, H. Valentin, J. Vernet, P. Weilbacher, L. Wisotzki, and N. Yerle. The MUSE second-generation VLT instrument. In *Society of Photo-Optical Instrumentation Engineers (SPIE) Conference Series*, volume 7735 of *Society of Photo-Optical Instrumentation Engineers (SPIE) Conference Series*, July 2010. doi: 10.1117/12.856027.

D. Fong and M. Saunders. LSMR: An iterative algorithm for sparse least-squares problems. *ArXiv e-prints*, June 2010.

S. Kamann, L. Wisotzki, and M. M. Roth. Resolving stellar populations with crowded field 3D spectroscopy. *A&A*, 549:A71, January 2013. doi: 10.1051/0004-6361/201220476.

L. Pasquini, G. Avila, A. Blecha, C. Cacciari, V. Cayatte, M. Colless, F. Damiani, R. de Propris, H. Dekker, P. di Marcantonio, T. Farrell, P. Gillingham, I. Guinouard, F. Hammer, A. Kaufer, V. Hill, M. Marteaud, A. Modigliani, G. Mulas, P. North, D. Popovic, E. Rossetti, F. Royer, P. Santin, R. Schmutzer, G. Simond, P. Vola, L. Waller, and M. Zoccali. Installation and commissioning of FLAMES, the VLT Multifibre Facility. *The Messenger*, 110:1–9, December 2002.

M. M. Roth, A. Kelz, T. Fechner, T. Hahn, S.-M. Bauer, T. Becker, P. Böhm, L. Christensen, F. Dionies, J. Paschke, E. Popow, D. Wolter, J. Schmoll, U. Laux, and W. Altmann. PMAS: The Potsdam Multi-Aperture Spectrophotometer. I. Design, Manufacture, and Performance. *PASP*, 117:620–642, June 2005. doi: 10.1086/429877.

R. M. Sharples, R. Bender, M. D. Lehnert, S. K. Ramsay Howat, M. N. Bremer, R. L. Davies, R. Genzel, R. Hofmann, R. J. Ivison, R. Saglia, and N. A. Thatte. KMOS: an infrared multiple-object integral

field spectrograph for the ESO VLT. In A. F. M. Moorwood and M. Iye, editors, *Ground-based Instrumentation for Astronomy*, volume 5492 of *Society of Photo-Optical Instrumentation Engineers (SPIE) Conference Series*, pages 1179–1186, September 2004. doi: 10.1117/12.550495.

A The PSF inspection file

Starting with version 0.92(?), PampelMuse offers the possibility to store detailed information about the PSF fits performed during CUBEFIT. This feature was implemented for the commissioning of the AO system of MUSE in June/July 2017. It only works if PampelMuse is run with the *smallifu* key set to False or if the PRM-file has been prepared using the SINGLESRC routine. Then, one can enable the key *inspect* in the *psf* section of the configuration. As a result, at the completion of CUBEFIT, an additional FITS file will be written containing detailed information about the fits of all PSF sources in all layers of the data cube.

psf|inspect	type=*boolean*	default=False	If enabled, an additional FITS file, named <prefix>.psf.fits will be written to disk at the end of the CUBEFIT routine, containing detailed information about the PSF stars and their fits. Note that this feature does not work in combination with the *smallifu* option of PampelMuse .

For each PSF star in the analysis (i.e. each star that has status 3 in the PRM-file), the FITS file will contain four HDUs with the following content.

- The values of the fitted PSF parameters in each layer that was analysed. The information is stored as a FITS-table, containing one column per fitted parameter and the wavelength information in the different rows. The names of these extensions are `PM_<source ID>`, where <source ID> is the ID of the star in extension `SOURCES` of the PRM-file.
- The radial profiles of the PSF stars in the IFS data as a function of wavelength. This information is stored as a two-dimensional array, containing the profile for one layer per column. The (approximate) wavelengths of the layers can be reconstructed from the `CRVAL1` and `CDELT1` keys in the header. The names of these extensions are `PR_<source ID>`, where again <source ID> is the ID of the star in extension `SOURCES` of the PRM-file. Note that the common radii of the individual points of all profiles (in spaxels) are provided as the data of the primary extension of the FITS-file.
- The two-dimensional profile of each PSF star extracted from the IFS data as a function of wavelength is stored as a mini data cube. Again, (approximate) dispersion information is encoded in the `CRVAL1` and `CDELT1` keys in the header. The names of these extensions are `DAT_<source ID>`.
- The two-dimensional residual maps of the PSF fits to every star as a function of wavelength are also stored as mini data cubes. The format is just the same as for the two dimensional profiles and the extensions are named `RES_<source ID>`.

B The PSF profiles

An overview of the functional forms of the analytical PSF profiles that PampelMuse uses. Besides the functional forms, we also provide the partial derivatives with respect to the individual profile parameters. They are used to determine the Jacobi matrix in the Levenberg-Marquard algorithm that performs the non-linear least squares fit.

B.1 Handling of positions

All profiles are defined as a function of r, with r giving the distance of a pixel (x_i, y_i) to the centroid. In the most general case, the PSF will be of elliptical shape, with its semi-major rotated by an angle θ **with respect to the y-axis**. For this reason, we define a coordinate system ($\hat{x}$, $\hat{y}$), that has the same origin as the PSF profile and has the $\hat{x}$-axis aligned with the semi-major axis. Assuming the central coordinates of profile n ($n \in [0, N)$) in the original system are (x^n, y^n), the transformation is defined as

$$\hat{x} = (x - x^n)\sin\theta - (y - y^n)\cos\theta\,, \tag{7}$$
$$\hat{y} = (x - x^n)\cos\theta + (y - y^n)\sin\theta\,. \tag{8}$$

The radius r is measured in units of pixels in direction of the semi-major axis of the profile, that is, for an arbitrary position angle θ with respect to that axis, r is calculated using

$$r(x, y) = \sqrt{\hat{x}^2 + \left(\frac{\hat{y}}{1-e}\right)^2}\,, \tag{9}$$

where e is the ellipticity of the profile.

The above equations imply that r is a function of $2N + 2$ parameters that can be free fit parameters in the PSF optimization: x^n, y^n, θ, and e. The partial derivatives of r with respect to those parameters are

$$\begin{aligned}\partial_{x^n} r \equiv \frac{\partial r}{\partial x^n} &= \left[\hat{x}^2 + \left(\frac{\hat{y}}{1-e}\right)^2\right]^{-1/2} \frac{1}{2}\left[2\hat{x}\frac{\partial \hat{x}}{\partial x^n} + \frac{2\hat{y}}{(1-e)^2}\frac{\partial \hat{y}}{\partial x^n}\right] \\ &= -\frac{1}{r}\left[\hat{x}\sin\theta + \frac{\hat{y}\cos\theta}{(1-e)^2}\right]\,, \end{aligned} \tag{10}$$

$$\begin{aligned}\partial_{y^n} r \equiv \frac{\partial r}{\partial y^n} &= \left[\hat{x}^2 + \left(\frac{\hat{y}}{1-e}\right)^2\right]^{-1/2} \frac{1}{2}\left[2\hat{x}\frac{\partial \hat{x}}{\partial y^n} + \frac{2\hat{y}}{(1-e)^2}\frac{\partial \hat{y}}{\partial y^n}\right] \\ &= \frac{1}{r}\left[\hat{x}\cos\theta - \frac{\hat{y}\sin\theta}{(1-e)^2}\right]\,, \end{aligned} \tag{11}$$

$$\begin{aligned}\partial_{\theta} r \equiv \frac{\partial r}{\partial \theta} &= \left[\hat{x}^2 + \left(\frac{\hat{y}}{1-e}\right)^2\right]^{-1/2} \frac{1}{2}\left[2\hat{x}\frac{\partial \hat{x}}{\partial \theta} + \frac{2\hat{y}}{(1-e)^2}\frac{\partial \hat{y}}{\partial \theta}\right] \\ &= \frac{\hat{x}\hat{y}}{r}\left[1 - \frac{1}{(1-e)^2}\right]\,, \end{aligned} \tag{12}$$

$$\begin{aligned}\partial_{e} r \equiv \frac{\partial r}{\partial e} &= \left[\hat{x}^2 + \left(\frac{\hat{y}}{1-e}\right)^2\right]^{-1/2} \frac{1}{2}\cdot\frac{2\hat{y}^2}{(1-e)^3}\,. \\ &= \frac{\hat{y}^2}{r\cdot(1-e)^3} \end{aligned} \tag{13}$$

However, the usual approach in PampelMuse is that we do not fit source coordinates directly, but that we assume they are already known in some reference frame and the task is to optimize the coordinate transformation from the reference system into the actual system of the datacube. Starting from the reference coordinates (u, v), we obtain the actual coordinates via

$$x^n = A\,u^n + B\,v^n + C\,, \tag{14}$$
$$y^n = A\,v^n - B\,u^n + D\,. \tag{15}$$

In this case, r is a function of 6 potentially free parameters: A, B, C, D, θ, and e. Accordingly, the partial derivatives $\partial_{x^n} r$ and $\partial_{y_n} r$ must be replaced by the partial derivatives with respect to A–D,

$$\begin{aligned}
\partial_A r \equiv \frac{\partial r}{\partial A} =& \partial_{x^n} r \cdot \frac{\partial x^n}{\partial A} + \partial_{y^n} r \cdot \frac{\partial y^n}{\partial A} = \partial_{x^n} r \cdot u^n + \partial_{y^n} r \cdot v^n \\
=& \frac{1}{r}\left[\hat{x} \cdot (v^n \cos\theta - u^n \sin\theta) - \frac{\hat{y} \cdot (u^n \cos\theta + v^n \sin\theta)}{(1-e)^2}\right], \qquad (16) \\
\partial_B r \equiv \frac{\partial r}{\partial B} =& \partial_{x^n} r \cdot \frac{\partial x^n}{\partial B} + \partial_{y^n} r \cdot \frac{\partial y^n}{\partial B} = \partial_{x^n} r \cdot v^n - \partial_{y^n} r \cdot u^n \\
=& -\frac{1}{r}\left[\hat{x} \cdot (u^n \cos\theta + v^n \sin\theta) + \frac{\hat{y} \cdot (v^n \cos\theta - u^n \sin\theta)}{(1-e)^2}\right], \qquad (17) \\
\partial_C r \equiv \frac{\partial r}{\partial C} =& \partial_{x^n} r \cdot \frac{\partial x^n}{\partial C} = \partial_{x^n} r\,, \qquad (18) \\
\partial_D r \equiv \frac{\partial r}{\partial D} =& \partial_{y^n} r \cdot \frac{\partial y^n}{\partial D} = \partial_{y^n} r\,. \qquad (19)
\end{aligned}$$

B.2 The Gaussian profile

The functional form of the Gaussian profile as it is used in PampelMuse is

$$G(r, e, \sigma, I) = \frac{I}{2\pi\sigma^2(1-e)} \exp\left\{\frac{-r^2}{2\sigma^2}\right\}. \qquad (20)$$

Note, however, that instead of the standard deviation σ, we use the FWHM to control the width of the Gaussian profile. The conversion between the two quantities is given as

$$\mathrm{FWHM} = \sqrt{8 \cdot \ln 2} \cdot \sigma\,. \qquad (21)$$

Furthermore, we control the integrated intensity I of the Gaussian (and any other) profile by providing a magnitude, i.e.,

$$I = 10^{-0.4 \cdot m}. \qquad (22)$$

The partial derivatives with respect to the individual parameters that define the shape of the profile (FWHM, σ, and θ) are calculated using the following formulae:

$$\begin{aligned}
\partial_{\mathrm{FWHM}} G &= \partial_\sigma G / \sqrt{8 \cdot \ln 2} \\
&= \exp\left\{\frac{-r^2}{2\sigma^2}\right\}\left(\frac{-I}{\pi\sigma^3(1-e)} + \frac{I \cdot r^2}{2\pi\sigma^5(1-e)}\right)/\sqrt{8 \cdot \ln 2} \\
&= G\left(\frac{r^2}{\sigma^3} - \frac{2}{\sigma}\right)/\sqrt{8 \cdot \ln 2}\,, \qquad (23) \\
\partial_e G &= \exp\left\{\frac{-r^2}{2\sigma^2}\right\}\left(\frac{I}{2\pi\sigma^2(1-e)^2} - \frac{I \cdot r \cdot \partial_e r}{2\pi\sigma^4(1-e)}\right) \\
&= G\left(\frac{1}{1-e} - \frac{r \cdot \partial_e r}{\sigma^2}\right), \qquad (24) \\
\partial_\theta G &= \frac{I}{2\pi\sigma^2(1-e)} \exp\left\{\frac{-r^2}{2\sigma^2}\right\} \cdot \frac{-r \cdot \partial_\theta r}{\sigma^2} \\
&= -G \cdot \frac{r \cdot \partial_\theta r}{\sigma^2}\,, \qquad (25)
\end{aligned}$$

where $\partial_e r$ and $\partial_\theta r$ are given as defined in Sect. B.1.

Regarding the partial derivatives with respect to any parameter connected to the centroid of the profile, the corresponding formulae given in Sect. B.1 are modified by a common factor,

$$\begin{aligned}
\partial_\nu G &= \frac{I}{2\pi\sigma^2(1-e)} \exp\left\{\frac{-r^2}{2\sigma^2}\right\} \cdot \frac{-r \cdot \partial_\nu r}{\sigma^2} \\
&= -G \cdot \frac{r \cdot \partial_\nu r}{\sigma^2}\,, \qquad (26)
\end{aligned}$$

with $\nu \in \{x^n, y^n, A, B, C, D\}$.

B.3 The Moffat profile

The functional form of the Moffat profile that is included in PampelMuse is given as

$$M(r, e, r_\mathrm{d}, \beta, I) = \frac{I \cdot (\beta - 1)}{\pi r_\mathrm{d}^2 \cdot (1 - e)} \left[1 + \left(\frac{r}{r_\mathrm{d}} \right)^2 \right]^{-\beta} , \tag{27}$$

where the width of the profile is mainly defined by the effective radius r_d and β controls the kurtosis of the profile. The integrated intensity I is the same as defined in Eq. 22. Similar to our approach for the Gaussian profile, we rather work with the FWHM instead of the effective radius r_d. The FWHM of the Moffat profile can be calculated via

$$\mathrm{FWHM} = 2\sqrt{2^{1/\beta} - 1} \cdot r_\mathrm{d} \, . \tag{28}$$

Equation 28 implies that the FWHM is a function of both, r_d and β. It should not be mixed up with the FWHM of a Gaussian profile that is a measure for the seeing during the observations. In general, the FWHM of a Moffat profile will be smaller than the seeing. This can be verified from Fig. 6, where we have plotted the FWHM ratio between a Gaussian and a Moffat profile as a function of β. The plotted ratios were obtained by fitting a Gaussian profile to the flux distribution created by a Moffat profile with given FWHM and β. Figure 6 shows that the shape of the Moffat profile approaches the shape of the Gaussian profile as β increases. Larger β values correspond to profiles with less pronounced wings. However, the Moffat profile will always have more pronounced wings than the Gaussian.

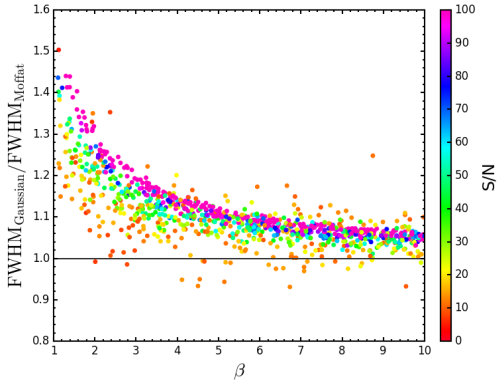


Figure 6: Inverse ratio between the FWHM of a Moffat profile and the FWHM of its best Gaussian fit, as a function of the β value of the Moffat profile and its S/N.

When calculating the partial derivatives of Eq. 27, we have to keep in mind that $r_\mathrm{d} = r_\mathrm{d}(\beta, \mathrm{FWHM})$, and therefore

$$\begin{aligned} \partial_\beta r_\mathrm{d} &= \frac{\mathrm{FWHM}}{2} \cdot \frac{-1}{2} \cdot (2^{1/\beta} - 1)^{-3/2} \cdot \frac{2^{1/\beta} \cdot \ln 2}{\beta^2} \\ &= \frac{r_\mathrm{d} \cdot \ln 2}{2 \cdot \beta^2 \cdot (1 - 2^{1/\beta})} \end{aligned} \tag{29}$$

$$\partial_\mathrm{FWHM} r_\mathrm{d} = \frac{1}{2\sqrt{2^{1/\beta} - 1}} \, . \tag{30}$$

With respect to the shape parameters FWHM, β, θ and e we thus obtain the partial derivatives

$$\begin{aligned}
\partial_{\mathrm{FWHM}} M &= \frac{-2I\cdot(\beta-1)}{\pi r_\mathrm{d}^3\cdot(1-e)}\cdot\partial_{\mathrm{FWHM}} r_\mathrm{d}\left[1+\left(\frac{r}{r_\mathrm{d}}\right)^2\right]^{-\beta} \\
&\quad - \frac{I\cdot\beta\cdot(\beta-1)}{\pi r_\mathrm{d}^2\cdot(1-e)}\left[1+\left(\frac{r}{r_\mathrm{d}}\right)^2\right]^{-\beta-1}\frac{-2r^2}{r_\mathrm{d}^3}\partial_{\mathrm{FWHM}} r_\mathrm{d} \\
&= \frac{2M}{\mathrm{FWHM}}\left[\beta\left(\frac{r}{r_\mathrm{d}}\right)^2\left[1+\left(\frac{r}{r_\mathrm{d}}\right)^2\right]^{-1} - 1\right], \qquad (31)\\
\partial_\beta M &= \left(\frac{I}{\pi r_\mathrm{d}^2\cdot(1-e)} - \frac{2I\cdot(\beta-1)}{\pi r_\mathrm{d}^3\cdot(1-e)}\cdot\partial_\beta r_\mathrm{d}\right)\left[1+\left(\frac{r}{r_\mathrm{d}}\right)^2\right]^{-\beta} \\
&\quad - \frac{I\cdot(\beta-1)}{\pi r_\mathrm{d}^2\cdot(1-e)}\left[1+\left(\frac{r}{r_\mathrm{d}}\right)^2\right]^{-\beta}\left(\ln\left[1+\left(\frac{r}{r_\mathrm{d}}\right)^2\right] + \frac{2^{1+1/\beta}\beta r^2}{r_\mathrm{d}^3\left[1+\left(\frac{r}{r_\mathrm{d}}\right)^2\right]}\partial_\beta r_\mathrm{d}\right) \\
&= M\left(\frac{1}{\beta-1} - \frac{2\partial_\beta r_\mathrm{d}}{r_\mathrm{d}} - \ln\left[1+\left(\frac{r}{r_\mathrm{d}}\right)^2\right] - \frac{2^{1+1/\beta}\beta r^2}{r_\mathrm{d}^3\left[1+\left(\frac{r}{r_\mathrm{d}}\right)^2\right]}\partial_\beta r_\mathrm{d}\right) \\
&= M\left(\frac{1}{\beta-1} - \ln\left[1+\left(\frac{r}{r_\mathrm{d}}\right)^2\right] - \frac{2\partial_\beta r_\mathrm{d}}{r_\mathrm{d}}\cdot\left[1+\frac{2^{1/\beta}\beta\cdot\left(\frac{r}{r_\mathrm{d}}\right)^2}{1+\left(\frac{r}{r_\mathrm{d}}\right)^2}\right]\right), \qquad (32)\\
\partial_\theta M &= \frac{-\beta\cdot I\cdot(\beta-1)}{\pi r_\mathrm{d}^2\cdot(1-e)}\left[1+\left(\frac{r}{r_\mathrm{d}}\right)^2\right]^{-\beta-1}\frac{2r}{r_\mathrm{d}^2}\cdot\partial_\theta r \\
&= -\frac{2M\cdot\beta\cdot r}{r_\mathrm{d}^2}\left[1+\left(\frac{r}{r_\mathrm{d}}\right)^2\right]^{-1}\cdot\partial_\theta r\,, \qquad (33)\\
\partial_e M &= \frac{I\cdot(\beta-1)}{\pi r_\mathrm{d}^2\cdot(1-e)^2}\left[1+\left(\frac{r}{r_\mathrm{d}}\right)^2\right]^{-\beta} \\
&\quad - \frac{\beta\cdot I\cdot(\beta-1)}{\pi r_\mathrm{d}^2\cdot(1-e)}\left[1+\left(\frac{r}{r_\mathrm{d}}\right)^2\right]^{-\beta-1}\frac{2r}{r_\mathrm{d}^2}\cdot\partial_e r \\
&= M\left(\frac{1}{1-e} - \frac{2\beta\cdot r\cdot\partial_e r}{r_\mathrm{d}^2}\left[1+\left(\frac{r}{r_\mathrm{d}}\right)^2\right]^{-1}\right). \qquad (34)
\end{aligned}$$

Just like in the case of the Gaussian profile, the partial derivatives with respect to the parameters that define the centroid of a profile can be obtained from the formulae in Sect. B.1 via multiplication with a constant factor:

$$\begin{aligned}
\partial_\nu M &= \frac{-\beta\cdot I\cdot(\beta-1)}{\pi r_\mathrm{d}^2\cdot(1-e)}\left[1+\left(\frac{r}{r_\mathrm{d}}\right)^2\right]^{-\beta-1}\frac{2r}{r_\mathrm{d}^2}\cdot\partial_\nu r \\
&= -\frac{2M\cdot\beta\cdot r}{r_\mathrm{d}^2}\left[1+\left(\frac{r}{r_\mathrm{d}}\right)^2\right]^{-1}\cdot\partial_\nu r\,, \qquad (35)
\end{aligned}$$

where again $\nu \in \{x^n, y^n, A, B, C, D\}$.